Forecasting a Café's Daily Ingredient Demand with Damped Triple Exponential Smoothing

1st Given Name Surname
Department of Informatics
Universität Hamburg
Hamburg, Germany
email address or ORCID

2nd Given Name Surname
Department of Informatics
Universität Hamburg
Hamburg, Germany
email address or ORCID

3rd Given Name Surname
Department of Informatics
Universität Hamburg
Hamburg, Germany
email address or ORCID

***Abstract*—We demonstrate a lightweight forecasting pipeline that predicts the daily ingredient demand of a café from its raw sales history. Per-drink sales records are loaded into a relational database, joined with recipe data that we resolve into six base ingredients, and aggregated into one daily time series per ingredient. A self-implemented additive Holt–Winters model with a damped trend extrapolates each series into the future. Visitors of the demo control the ingredient, the forecast window, and all four smoothing parameters through a command line interface and receive an automatically generated plot within a second, making the effect of every parameter directly visible.**

***Index Terms*—forecasting, time series, triple exponential smoothing, Holt–Winters, damped trend, PostgreSQL**

I. Introduction

A café records *what it sells* – lattes, cappuccinos, cocoa – but it has to purchase *what it consumes*: coffee beans, milk, water, cocoa powder, sugar, and salt. Purchasing therefore requires two steps the raw sales log does not provide: translating drinks into ingredient quantities, and anticipating future demand. Both matter economically: perishables such as milk spoil when overstocked, while an empty bean hopper halts the business. This pattern – transactional records on one level, planning needs on another – appears in virtually every retail scenario, making ingredient-level forecasting a broadly useful building block.

Our demo solves this end to end for a one-year sales log of a café (3 547 transactions, March 2024 to March 2025) provided as course material[1]. Sales and recipes are integrated in a PostgreSQL database, aggregated into daily ingredient-usage series, and forecast by an additive Holt–Winters model with a damped trend that we implemented from scratch. A command line tool exposes every model parameter, so visitors can explore – and break – the forecast interactively.

II. Architecture Overview

Figure 1 shows the pipeline. The raw sales file `Coffe_sales.csv` is copied unchanged into the relational table `coffee_sales` in PostgreSQL; the CSV is never read again afterwards, all queries run against the database.

The recipes originate from the completed JSON document collection of the second exercise sheet. Instead of querying a document store at runtime, we hard-code the completed recipes as a constant in the loader script: the recipe set is tiny (eight drinks), static, and needed in full on every run, so a second database system would add operational complexity without benefit. While loading, the pseudo-ingredient *espresso shot* is recursively resolved into its base ingredients (18 g coffee beans and 30 ml water per shot), and where a recipe offers "milk or water" (Cocoa) we assume *water*. The result is stored in the table `recipe_base_ingredients(drink_name, base_ingredient, amount)`. A SQL view `daily_ingredient_usage` joins it with the sales table and aggregates per day and ingredient, so the entire drink-to-ingredient translation happens declaratively inside the database. The Python tool extracts one ingredient's series from this view, fills days without sales with zero to obtain a gap-free daily series, forecasts it, and renders the plot with matplotlib.

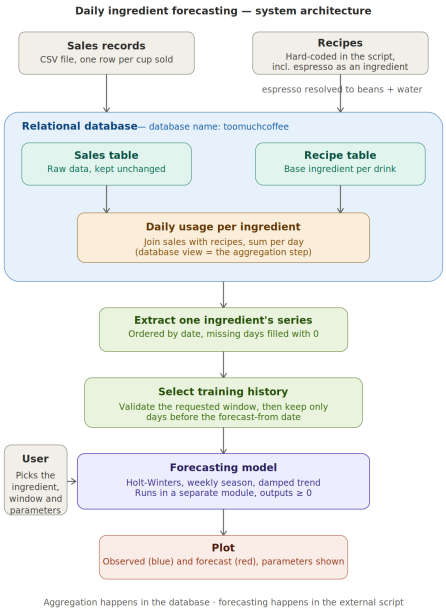

Fig. 1. Architecture of the demo: sales and recipe data are integrated in PostgreSQL, aggregated by a view, and consumed by the Python forecasting tool.

[1]Datasets *Coffee sales* and *Coffee recipes* from the course *Databases and Information Systems*, Universität Hamburg, summer term 2026.

A. Forecasting Approach

We use additive triple exponential smoothing (the Holt–Winters method) [2], [3] with a season length of $m = 7$ days, since the data shows a clear weekly rhythm, extended by a damped trend [4]. With smoothing factors $\alpha, \beta, \gamma \in [0, 1]$ and damping factor $\phi \in (0, 1]$, level ℓ_t, trend b_t, and season s_t are updated for every observation x_t as

$$\ell_t = \alpha(x_t - s_{t-m}) + (1 - \alpha)(\ell_{t-1} + \phi b_{t-1}), \quad (1)$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)\,\phi b_{t-1}, \quad (2)$$

$$s_t = \gamma(x_t - \ell_t) + (1 - \gamma)\,s_{t-m}, \quad (3)$$

and the h-step-ahead forecast is $\hat{x}_{T+h} = \ell_T + (\sum_{i=1}^{h} \phi^i)\, b_T + s_{T-m+1+((h-1) \bmod m)}$. Damping ($\phi < 1$) lets the trend level off to a plateau instead of extrapolating linearly, which keeps long-horizon forecasts plausible; $\phi = 1$ recovers the classical model. Level, trend, and seasonal indices are initialized naively from the first two weeks of data, and all forecast values are clipped to be non-negative, since negative ingredient usage is impossible. The recursion is implemented in pure Python without any forecasting library.

III. Demonstration Proposal

The demo is an interactive command line session in front of live plots. Visitors first see a prepared forecast, then change parameters themselves and immediately observe how the model reacts. This section describes the technical setup and the guided walk-through.

A. Setup

The demo runs locally on a laptop (AMD Ryzen 7 7840HS, 32 GB main memory) under Ubuntu 24.04, with PostgreSQL 16.14 and Python 3.12.3 using `psycopg2`, `argparse`, and `matplotlib`. No network connection is required; a full run – loading, aggregating, forecasting, and plotting – completes in well under a second.

B. User Experience

Visitors interact with the tool through command line flags: `-i` switches between the six ingredients, `--forecast_from` and `--forecast_until` set the inclusive forecast window, and `-a`, `-b`, `-g`, `-p` control α, β, γ, and ϕ. The plot opens automatically after every run. Implausible inputs, such as a window starting before the observed data or smoothing factors outside $[0, 1]$, are rejected with a descriptive message.

Two scenarios structure the walk-through. First, *validation*: the window is placed inside the observed range, so the model only sees the history before the window and its prediction can be compared visually against reality (Fig. 2) – the weekly seasonality is clearly reproduced. Second, *looking ahead*: the window is placed after the last observed day (Fig. 3). Visitors are then invited to experiment, e.g., to switch to milk, to set $\phi = 1$ and watch the undamped trend drift over a long horizon, or to set $\alpha = 1$ and see the forecast chase noise.

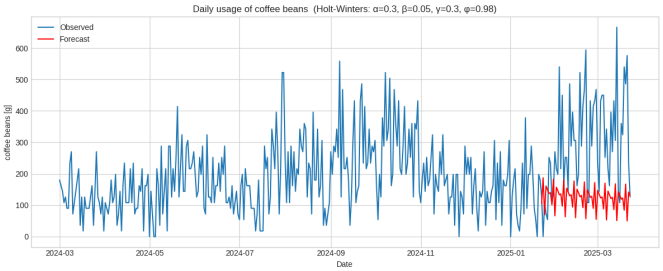

Fig. 2. Validation: forecast (red) of the last two observed months of coffee-bean usage, computed only from the history before the window (α=0.3, β=0.05, γ=0.3, ϕ=0.98).

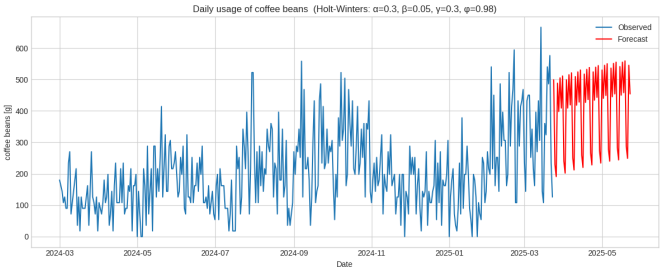

Fig. 3. Looking ahead: two-month forecast beyond the observed data, same parameters as in Fig. 2.

C. Conclusion

The demo shows that a complete ingredient-demand forecasting pipeline needs little machinery: one relational database, one SQL view for the drink-to-ingredient translation, and roughly one hundred lines of self-written Holt–Winters code. Its distinguishing feature is full parameter transparency – every smoothing factor is a command line flag, turning the demo into a hands-on playground for exponential smoothing.

Acknowledgment

Parts of the source code were developed with the assistance of Anthropic's large language model Claude and reviewed by the authors.

References

[1] C. C. Holt, "Forecasting seasonals and trends by exponentially weighted moving averages," *International Journal of Forecasting*, vol. 20, no. 1, pp. 5–10, 2004, reprint of ONR Research Memorandum 52, Carnegie Institute of Technology, 1957.

[2] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," *Management Science*, vol. 6, no. 3, pp. 324–342, 1960.

[3] E. S. Gardner, Jr. and E. McKenzie, "Forecasting trends in time series," *Management Science*, vol. 31, no. 10, pp. 1237–1246, 1985.